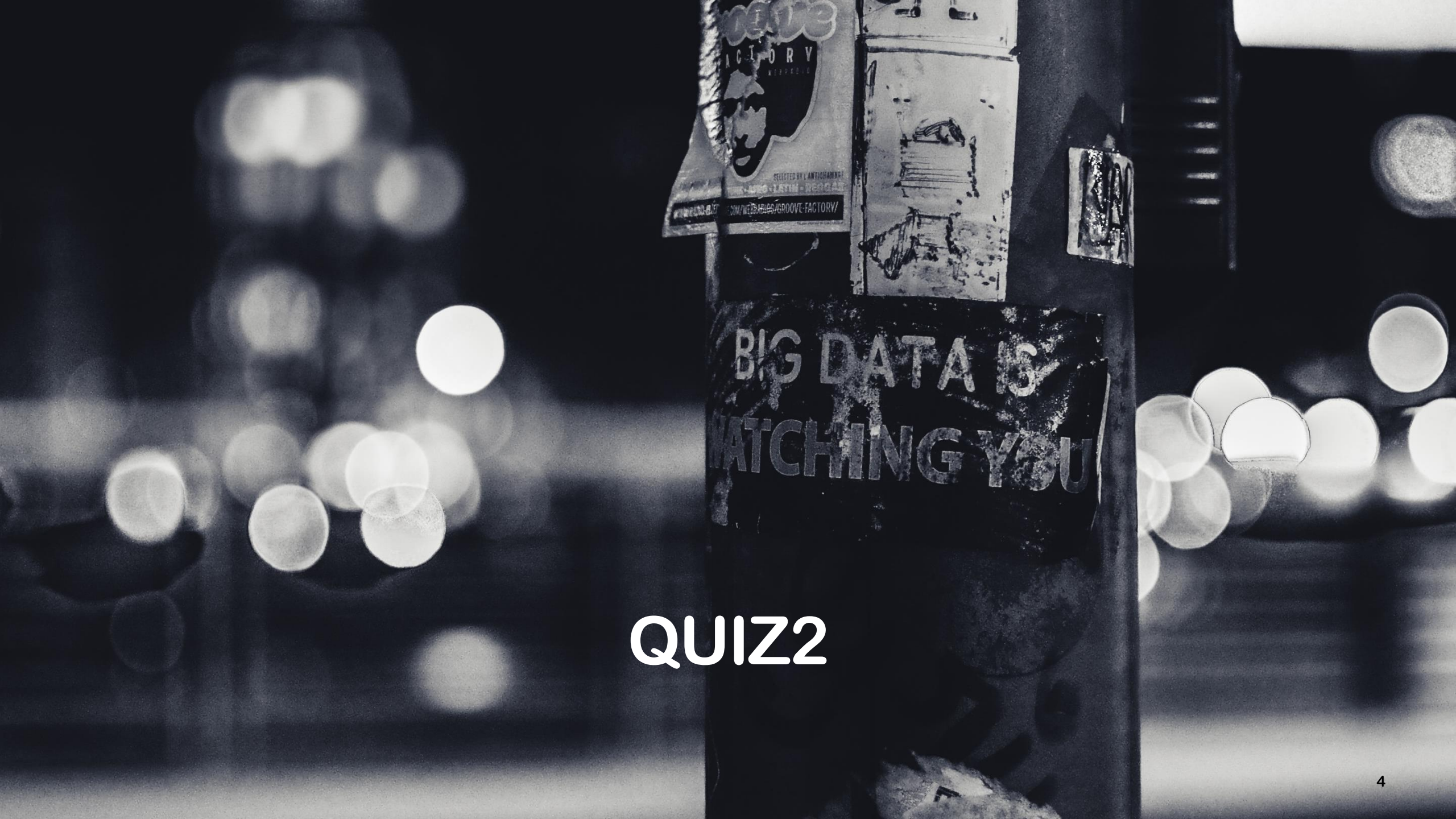


WEEK8 - COS20028 (BIG DATA ARCHITECTURE AND APPLICATION)

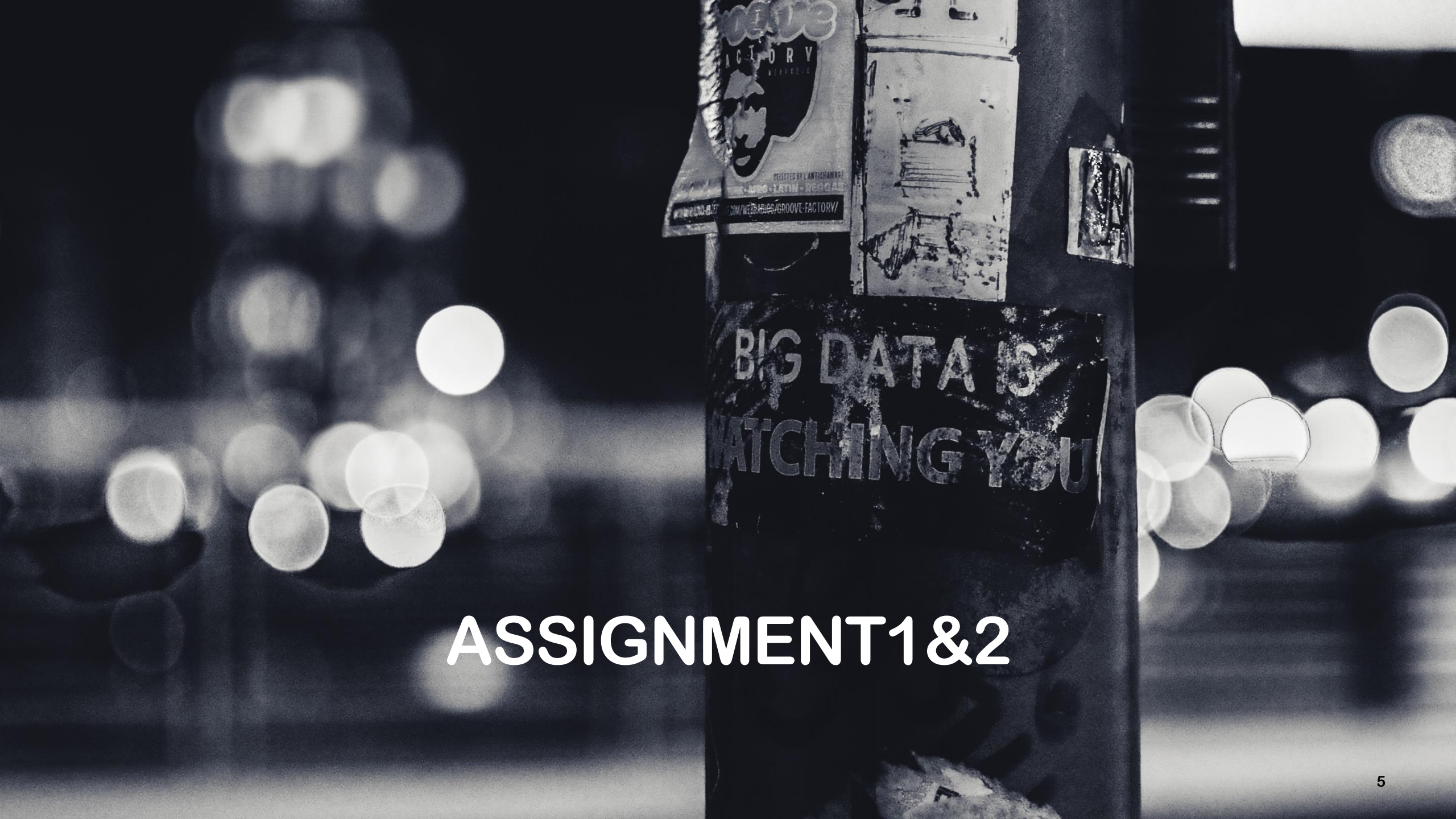
Dr Pei-Wei Tsai (Unit Convenor)
Yicun Tian

Week 7 Overview: What You'll Learn

Source	Content
Online Videos (Canvas)	Part 1: Apache Sqoop for Data Acquisition Part 2: Apache Flume Part 3: Apache Oozie Part 4: Apache Kafka
Live Session	Recap
Lab Activities	1. Exploring a Secondary Sort Example 2. Sqoop 3. Kafka



QUIZ2



BIG DATA IS
WATCHING YOU

ASSIGNMENT1&2



RECAP – SQOOP

Background

- RDBMS stands for Relational Database Management System
- It stores data in **tables** made up of **rows** and **columns**
- Common RDBMS products include:
 - MySQL, PostgreSQL, Oracle, SQL Server, etc.
- It supports **structured queries** using **SQL**

In the Hadoop ecosystem, a lot of raw data is not originally stored in HDFS.

Instead, it is often stored in traditional relational databases — also known as RDBMS (Relational Database Management Systems). -

This is a very common scenario:

You want to use the data stored in an RDBMS for your MapReduce analysis tasks.

So...

What can you do?

How do you get data from a database into your MapReduce job?

RDBMS → HDFS

Directly Reading from RDBMS (Not Recommended)

- Although it is technically possible to read from a database directly inside your Mapper, this approach is strongly discouraged.
- Why?
 - Multiple Mappers run in parallel, which leads to a large number of concurrent database connections.
 - This is equivalent to performing a DDoS (Distributed Denial of Service) attack on the database.
 - It can severely impact the stability and performance of the database server.
- What you need to understand:
 - Never connect to a database directly within a MapReduce job to read data.
 - It can overwhelm and crash the database server.

RDBMS → HDFS

A Better Way: Use Sqoop to Import Data into HDFS

- ◆ Sqoop is the recommended tool for efficiently importing structured data from relational databases (RDBMS) into Hadoop's file system (HDFS).
- ◆ After importing, the data can be processed using MapReduce, Hive, Spark, or other Hadoop ecosystem tools.

Concept	Explanation
RDBMS	Acts as the source of structured, transactional data
HDFS	The storage layer in Hadoop, optimized for large-scale data processing
Sqoop	A data transfer tool that imports RDBMS tables into HDFS efficiently

Sqoop - outline

- **Understand What Sqoop Is**
- **Explain How Sqoop Works**
- **Use Basic Sqoop Syntax**
- **Choose File Format and Handle Data**
- **Perform Incremental Imports**
- **Understand Custom Connectors**

Explain How Sqoop Works

1. Built on MapReduce

- Sqoop uses MapReduce jobs to perform the actual data import from relational databases.
- Each Mapper handles a split of the input data.
- Parallel import improves performance and scalability for large datasets.

2. Parallelism with Configurable Mappers

- **Default: 4 Mappers**
- Can be adjusted using: `--num-mappers <N>`
- Throttles the number of mappers to prevent overloading the source database.
 - Avoids DDoS-like behavior from too many concurrent connections.

3. JDBC Interface for Connectivity

- Connects to databases using standard **JDBC drivers**.
- Compatible with most relational databases: MySQL, PostgreSQL, Oracle, SQL Server, etc.
- Ensure the correct JDBC driver .jar is available in the lib/ directory.

Use Basic Sqoop Syntax

Section	Description	Example / Note
Syntax	General command format	<code>sqoop <tool-name> [tool-options]</code>
Tool Name	Operation to perform	<code>import, import-all-tables, list-tables</code>
Connection	Specifies how to connect to the database	<code>--connect jdbc:mysql://host/dbname</code>
User Auth	Login credentials	<code>--username root, --password secret</code>
Table Name	Which table to import	<code>--table employees</code>
Target Dir	HDFS output directory	<code>--target-dir /user/hadoop/employees</code>
Mappers	Number of parallel import tasks	<code>--num-mappers 2 (default is 4)</code>
Optional	Incremental load, column selection, file format	<code>--incremental append, --columns, --as-avrodatafile</code>

Example

```
sqoop import \  
  --connect jdbc:mysql://localhost/employees \  
  --username root \  
  --password secret \  
  --table staff \  
  --target-dir /user/hadoop/staff \  
  --num-mappers 2
```

Part	Explanation
sqoop import	Use the import tool to bring data into HDFS
--connect	JDBC connection string to the MySQL database
--username / --password	Credentials for database login
--table staff	The table to be imported (in this case, staff)
--target-dir	HDFS path where the data will be saved
--num-mappers 2	Number of parallel Map tasks used for import

Choose File Format and Handle Data

Feature	Description	Example / Default
File Format	Format used to store data in HDFS	Default: Delimited text file (CSV) Options: --as-sequencefile, --as-avrodatafile, --as-parquetfile
Incremental Import	Import only new rows since the last import	Use: --incremental append, --check-column id, --last-value 100
Generated Java Class	Sqoop creates a class to represent each row of data (for MR jobs)	Use: --bindir /path/to/save/class
Delimiter Customization	Change field/line separators in text files	Use: --fields-terminated-by ','

Example

```
sqoop import \  
  --connect jdbc:mysql://localhost/employees \  
  --username root \  
  --password secret \  
  --table sales \  
  --target-dir /data/sales \  
  --as-avrodatafile \  
  --incremental append \  
  --check-column id \  
  --last-value 300
```

```
{  
  "type": "record",  
  "name": "employee",  
  "fields": [  
    {"name": "id", "type": "int"},  
    {"name": "name", "type": "string"}  
  ]  
}
```

Understand Custom Connectors

What Are Custom Sqoop Connectors?

Aspect	Description
Definition	Special connectors developed by Cloudera and partners to connect specific enterprise databases using native protocols rather than JDBC.
Purpose	Provide faster and more optimized data transfer compared to general JDBC-based connectors.
Availability	Free to use, but not open source. Must be downloaded separately.
Where to Get	Available from Cloudera's website or official partners.

- **Why It Matters**
- JDBC is **generic** and may be slower with large volumes.
- Custom connectors use **native APIs**, offering:
 - Higher throughput
 - Better handling of metadata types
 - Improved reliability for enterprise-scale jobs

T/F

1. Sqoop uses MapReduce jobs to perform data import and export operations.
2. Sqoop can only import entire tables and does not support importing a subset of data.
3. By default, Sqoop uses four Mappers to parallelize the import process.
4. Sqoop requires a JDBC connection to access relational databases.
5. The --as-avrodatafile option in Sqoop exports data as plain text in CSV format.
6. Custom Sqoop connectors provide optimized performance but are only available for a fee.

T/F

Sqoop uses MapReduce jobs to perform data import and export operations.

True → Sqoop leverages MapReduce to parallelize the data transfer process.

2. Sqoop can only import entire tables and does not support importing a subset of data.

False → Sqoop supports filtering using options like --where and selecting specific columns with --columns.

3. By default, Sqoop uses four Mappers to parallelize the import process.

True → The default number of mappers is 4, but this can be changed using --num-mappers.

4. Sqoop requires a JDBC connection to access relational databases.

True → JDBC is the standard way Sqoop connects to databases like MySQL, PostgreSQL, etc.

5. The --as-avrodatafile option in Sqoop exports data as plain text in CSV format.

False → This option stores data in the Avro binary format, which is compact and schema-aware.

6. Custom Sqoop connectors provide optimized performance but are only available for a fee.

False → Custom connectors are not open source, but they are free to use, especially from Cloudera.



BIG DATA IS
WATCHING YOU

RECAP – KAFKA

Kafka Learning Outline

1. What is Kafka?

- Definition: A distributed event streaming platform
- Differences between Kafka and traditional message queues
- Real-world use cases: e-commerce, finance, IoT, log aggregation, etc.

2. Kafka Architecture - Core Components:

- Producer – sends messages
- Consumer – reads messages
- Topic – logical channel for categorizing messages
- Partition – enables parallelism and scalability
- Offset – unique identifier of a message's position in a partition
- Broker – Kafka server that stores and serves messages
- Consumer Group – enables load balancing and parallel consumption
- Replication, Leader, Follower – fault-tolerance mechanisms

Understand how Kafka achieves high availability and scalability

Kafka Learning Outline

3. Basic Operations & CLI

- How to create a topic
- How to send and consume messages(using kafka-console-producer.sh / kafka-console-consumer.sh)
- How to check topic, partition, and replica information
- Summary of commonly used commands

◦ 4. Kafka in Action

- Typical project scenarios: order processing, logging systems, alert pipelines
- Kafka message flow: Producer → Topic → Consumer

5. Why Kafka Matters

- Why Kafka is widely adopted in the industry
- Kafka's role in microservices, data pipelines, and real-time architectures

Why Learn Kafka?

Kafka Is Everywhere

- Kafka is used by major companies around the world to stream, process, and connect massive volumes of data in real time.
- Kafka is a "distributed event streaming platform."
- It can be considered a tool, but it is also often referred to as a framework, a system, or even a platform.

Reason	Explanation
Real-Time Data	Kafka enables instant processing of logs, orders, events, and user actions.
High Throughput & Scalability	Capable of handling millions of messages per second across clusters.
System Decoupling	Allows systems to communicate asynchronously with minimal dependencies.
Core of Modern Architectures	Foundational in microservices, big data, and event-driven systems.
Industry Demand	Kafka skills are in high demand in software, data engineering, and DevOps roles.